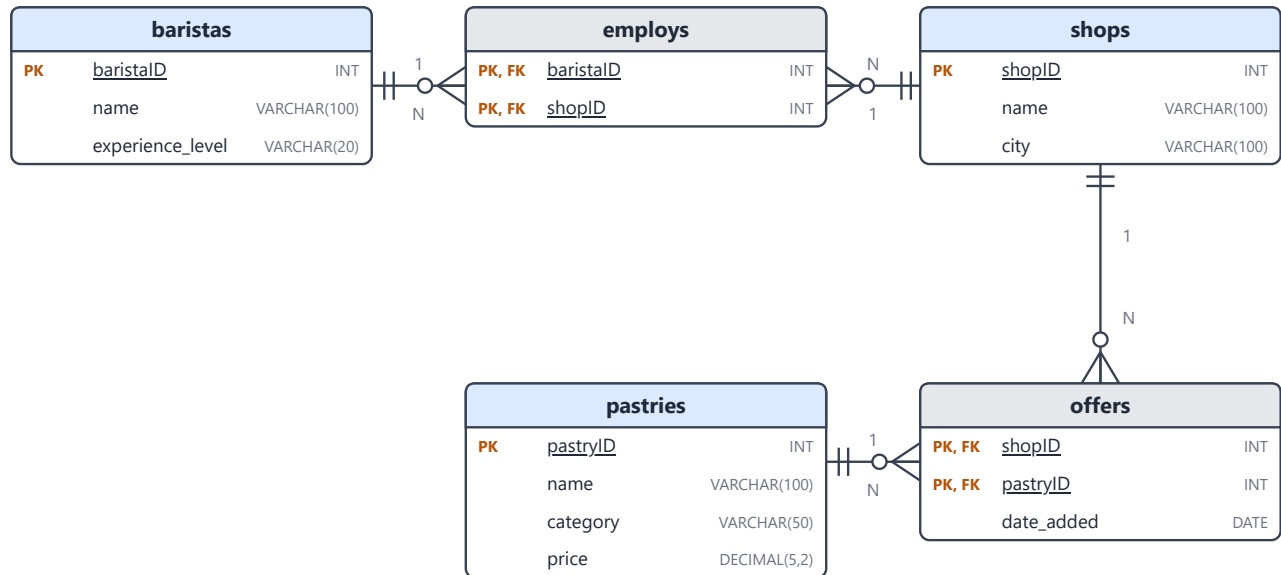


HW2 — Assignment 2

Hien Le · Database Management · Coffee Shop database (MySQL)

Entity-Relationship Diagram



PK primary key **FK** foreign key **PK, FK** part of a composite primary key and a foreign key
Notation: || = exactly one, crow's foot with circle = zero or many. Shaded grey tables are junction tables.

- **shops**, **baristas** and **pastries** each have a single-column primary key (shopID, baristaID, pastryID).
- **employs** resolves the many-to-many relationship between baristas and shops. Its composite primary key is (baristaID, shopID); both columns are foreign keys. A barista can work at several shops, and a shop can employ several baristas.
- **offers** resolves the many-to-many relationship between shops and pastries. Its composite primary key is (shopID, pastryID); both columns are foreign keys, and date_added records when a shop began offering the pastry.

SQL Queries and Results

Problem 1

Find the average price of pastries for each category from the pastries table.

```
select category, avg(price) as avg_price
from pastries
group by category;
```

Result (MySQL Workbench)

	category	avg_price
▶	Pastry	4.666666666666667
	Muffin	3.625
	Bakery	4.25
	Scone	3.85
	Breakfast	8
	Cake	3.875

result1 × result2 result3 res

Problem 2

Find the total number of baristas at each experience level from the baristas table.

```
select experience_level, count(*)
from baristas
group by experience_level;
```

Result (MySQL Workbench)

	experience_level	count(*)
▶	Junior	2
	Senior	2
	Mid-Level	2

result1 result2 × result3 r

Problem 3

Count the total number of shops located in each city from the shops table.

```
select city, count(*) as num_shops
from shops
group by city;
```

Result (MySQL Workbench)

	city	num_shops
▶	Seattle	1
	Portland	1
	Austin	1
	Chicago	1
	Denver	1
	Boston	1

result1 result2 result3 × res

Problem 4

Find the maximum price among pastries for each category from the pastries table.

```
select category, max(price) as max_price
from pastries
group by category;
```

Result (MySQL Workbench)

	category	max_price
▶	Pastry	5.5
	Muffin	3.75
	Bakery	4.25
	Scone	3.85
	Breakfast	8.5
	Cake	4

result1 result2 result3 result4 × result5

Problem 5

Count how many pastries have been added by each shop using the shopID column from the offers table.

```
select shopID, count(pastryID) as num_pastries
from offers
group by shopID;
```

Result (MySQL Workbench)

	shopID	num_pastries
▶	1	2
	2	2
	3	2
	4	2
	5	2
	6	2

result1 result2 result3 result4 **result5** × result6

Problem 6

Find the name, category, and price of any pastry whose price matches the maximum price within its category.

```
SELECT p.name, p.category, p.price
FROM pastries p
WHERE p.price = ( SELECT MAX(p2.price)
                  FROM pastries p2
                  WHERE p2.category = p.category);
```

Result (MySQL Workbench)

	name	category	price
▶	Almond Croissant	Pastry	5.5
	Blueberry Muffin	Muffin	3.75
	Cinnamon Roll	Bakery	4.25
	Chocolate Scone	Scone	3.85
	Blueberry Scone	Scone	3.85
	Breakfast Burrito	Breakfast	8.5
	Lemon Loaf	Cake	4

result1 result2 result3 result4 result5 **result6** × result7

Problem 7

Find the unique shop IDs from the offers table that have offered at least one pastry whose price is strictly greater than the overall average price of all pastries.

```
SELECT DISTINCT shopID
FROM offers
WHERE pastryID IN ( SELECT pastryID
                    FROM pastries
                    WHERE price > ( SELECT AVG(price)
                                    FROM pastries));
```

Result (MySQL Workbench)

	shopID
▶	1
	5

result1result2result3result4result5result6result7 × result8

Problem 8

Find the shop ID and pastry ID for the records in the offers table that have the earliest date_added (minimum date).

```
SELECT shopID, pastryID
FROM offers
WHERE date_added = (SELECT MIN(date_added)
                   FROM offers);
```

Result (MySQL Workbench)

	shopID	pastryID
▶	1	1

result1result2result3result4result5result6result7result8 × result9

Problem 9

Find the shop ID(s) that offer the highest number of pastries, utilizing a subquery to evaluate the maximum count per shop.

```
SELECT shopID, COUNT(*) as num_pastries
FROM offers
GROUP BY shopID
HAVING COUNT(*) IN (SELECT MAX(cnt)
                    FROM ( SELECT COUNT(*)
                          AS cnt FROM offers
                          GROUP BY shopID) AS t);
```

Result (MySQL Workbench)

	shopID	num_pastries
▶	1	2
	2	2
	3	2
	4	2
	5	2
	6	2

result1result2result3result4result5result6result7result8result9 ×result10

Problem 10

Find the names of baristas who work at shops located in 'Seattle' using nested subqueries.

```
SELECT name
FROM baristas
WHERE baristaID IN (select baristaID
                    from employs
                    where shopID in (select shopID
                                    from shops
                                    where city = "Seattle"));
```

Result (MySQL Workbench)

	name
▶	Emma Watson
	Liam Chen

result1result2result3result4result5result6result7result8result9result10 ×